SmartStay – Viva Presentation & Architecture Guide

SLIIT Software Engineering (SE2030) | Year 2 Semester 1 | Group: Y2-S1-MLB-WEB1G1-07

Group Member & Functional Responsibility Allocation

Student ID	Team Member Name	Assigned Major Function	Primary Key User
IT25100433	Induwara.S.S.R.D.B.T	**1. Room Management**	Front Office / Guest
IT25100436	Kathriarachchi.K.P	**2. Reservation Management**	Reservations Officer / Guest
IT25100447	Perera.U.V.G.C	**3. Payment and Billing Management**	Finance / Reservations / Guest
IT25100457	Perera.P.P.S	**4. Customer (Guest) Management**	Guest / Guest Relations
IT25100464	Wijesiri.W.D.H.A.S	**5. Feedback and Complaint Management**	Guest Relations / Manager
IT25100470	FernandoB.V.I	**6. User and Staff Management**	IT Coordinator / Admin

System-Wide Common Architecture (Shared Across All 6 Members)

Topic	Quick Viva Explanation & Exact Code References
Database Connection	Configured in **`src/main/resources/application.properties:L15-L25`**. Connects to **Microsoft SQL Server** (`SmartStayDB`) via `mssql-jdbc` using **HikariCP** (10 max pool size). Data access uses Spring's `JdbcTemplate` with parameterized SQL (`?` placeholders) preventing SQL injection and avoiding heavy ORM lazy-loading anomalies.
UI Demo: Button Color	File: **`src/main/resources/static/css/main.css:L63`**. Change `.btn-primary { background: #2563eb !important; }` (Royal Blue) or `#dc2626 !important;` (Red). Or change CSS variables `--accent` and `--accent-hover` at lines 12–13.
UI Demo: Author Details	File: **`src/main/resources/templates/fragments/footer.html:L66`**. Replace static copyright text with your team member details: `© 2026 SmartStay | Developed by [Name] ([IT Number]) - SLIIT SE2030`.
Design Patterns	**MVC** (Spring Controller + Thymeleaf View + Model/DTO), **Repository/DAO** (`JdbcTemplate` query isolation), **Service Layer** (`@Transactional` domain rules), **DTO** (over-posting defense), **Interceptor** (`AuthInterceptor` RBAC), **Singleton** (Spring bean scope), and **Strategy/Template Method** (`RowMapper<T>`).

FUNCTION 1: ROOM MANAGEMENT

Assigned Member: IT25100433 – Induwara.S.S.R.D.B.T | **Main Users:** Front Office Manager, Reservations Officer, Guest

1. CRUD Operations & Complex Business Logic

- **Primary Files:** `com/smartstay/controller/RoomController.java`, `com/smartstay/service/RoomService.java`, `com/smartstay/repository/RoomRepository.java`
- **CREATE:** `RoomRepository.save(Room)` (Lines 137–152) inserts new room records. Associated facilities and gallery photos are saved in child tables `RoomFacility` and `RoomImage`.
- **READ (Dynamic Filter Search):** `searchRooms(RoomFilterDto)` in `RoomRepository.java:L78-L118` dynamically builds SQL query based on user filters (Branch, Room Type, Max Price, and Date Availability).

```
// Complex SQL availability filter inside searchRooms:
AND r.status = 'AVAILABLE'
AND r.room_id NOT IN (
    SELECT res.room_id FROM Reservation res
    WHERE res.status NOT IN ('CANCELLED')
      AND (res.check_in_date < ? AND res.check_out_date > ?)
)
```

- **UPDATE:** `updateStatus(roomId, status)` (`RoomRepository.java:L163`) updates room readiness (`AVAILABLE`, `OCCUPIED`, `MAINTENANCE`).
- **DELETE:** `delete(roomId)` removes rooms with automatic cascading deletion of `RoomImage` and `RoomFacility` records.
- **Complex Logic:** Live room readiness state management across branches and multi-condition dynamic SQL query generation.

2. Data Validations (Frontend & Backend)

- **Frontend:** Number range filter for price on `templates/rooms/list.html`; branch dropdown required.
- **Backend:** DDL constraint enforces `CHECK (status IN ('AVAILABLE','OCCUPIED','MAINTENANCE'))`; composite unique constraint `UNIQUE (branch_id, room_number)` prevents duplicate room numbers in the same hotel branch.

3. Database Schema & Tables

- `Room` (PK: `room_id`, FK: `branch_id` → `Branch`, `room_number`, `type`, `price`, `status`).
- `RoomImage` (PK: `image_id`, FK: `room_id` → `Room` ON DELETE CASCADE, `image_url`).
- `RoomFacility` (PK: `facility_id`, FK: `room_id` → `Room` ON DELETE CASCADE, `facility_name`).
- `Room_Promotion` (Composite PK: `(room_id, promo_id)` → Many-to-Many junction).

4. Design Patterns & Justification

- **Repository Pattern:** All complex SQL strings and `RowMapper` mappings for Room entities are encapsulated inside `RoomRepository`, keeping `RoomService` completely decoupled from JDBC queries.
- **DTO Pattern:** `RoomFilterDto` groups all filter criteria together for clean HTTP form binding.

FUNCTION 2: RESERVATION MANAGEMENT

Assigned Member: IT25100436 – Kathriarachchi.K.P | **Main Users:** Guest, Reservations Officer, Front Office Manager

1. CRUD Operations & Complex Business Logic

- **Primary Files:** `com/smartstay/controller/ReservationController.java`, `com/smartstay/service/ReservationService.java`, `com/smartstay/repository/ReservationRepository.java`
- **CREATE (Lines 68–192 in `ReservationService`):**
  - **Authoritative Double-Booking Check:** Calls `isRoomAvailable(...)` in `RoomRepository.java:L120-L135` with interval query:

    ```
    SELECT COUNT(*) FROM Reservation
    WHERE room_id = ? AND status NOT IN ('CANCELLED')
      AND (? IS NULL OR reservation_id <> ?)
      AND (check_in_date < ? AND check_out_date > ?)
    ```

  - **Dynamic Promotional Pricing:** `nights = ChronoUnit.DAYS.between(checkIn, checkOut); totalAmount = (room.price * nights) - discount;`
  - **Auto Invoice Ledger:** Generates initial `Invoice` statement with `total_amount` and `outstanding_balance`.
  - **Special Requests:** Records optional guest add-on requests into `SpecialRequest` table.
- **READ:** `getReservationById(...)` populates composite invoice, payments, and special requests.
- **UPDATE:** `modifyReservation(...)` (Lines 195–240) recalculates nights, pricing, and balances while excluding the current reservation from collision checks.
- **DELETE / CANCEL:** `cancelReservation(...)` (Lines 242–294) sets status to `CANCELLED`, frees room back to `AVAILABLE`, and automatically initiates a `Feedback` + `Refund` record if payments were verified!
- **Check-in / Check-out Transitions:** `checkIn()` moves room to `OCCUPIED`; `checkOut()` automatically moves room to `MAINTENANCE` for cleaning.

2. Data Validations (Frontend & Backend)

- **Frontend (`booking.js` & `validation.js`):** Sets check-in `min` to today, dynamically auto-advances check-out date to check-in + 1 day, enforces guest count 1–10.
- **Backend (`ReservationRequestDto.java`):** `@NotNull`, `@Min(1)`, `@DateTimeFormat(pattern="yyyy-MM-dd")`, plus service-level check `checkOut.isAfter(checkIn)` and double-booking conflict rejection.
- **Database Constraints:** `CHECK (check_out_date > check_in_date)`, `CHECK (status IN ('CONFIRMED','MODIFIED','CANCELLED','CHECKED_IN','CHECKED_OUT'))`.

3. Database Schema & Tables

- `Reservation` (PK: `reservation_id`, FK: `guest_id`, FK: `room_id`, FK: `entered_by_staff_id`).
- `SpecialRequest` (PK: `request_id`, FK: `reservation_id` ON DELETE CASCADE, `type`, `description`).

4. Design Patterns & Justification

- **Service Layer Pattern with `@Transactional`:** Atomic creation of reservation, invoice, and initial payment in a single database transaction. If any step fails, the entire transaction rolls back.
- **DTO Pattern:** `ReservationRequestDto` prevents mass assignment and binds request parameters safely.

FUNCTION 3: PAYMENT AND BILLING MANAGEMENT

Assigned Member: IT25100447 – Perera.U.V.G.C | **Main Users:** Guest, Reservations Officer, Finance / Admin

1. CRUD Operations & Complex Business Logic

- **Primary Files:** `com/smartstay/controller/PaymentController.java`, `com/smartstay/service/PaymentService.java`, `com/smartstay/repository/PaymentRepository.java`, `com/smartstay/repository/InvoiceRepository.java`
- **CREATE (Record Payment):** `recordPayment(PaymentRequestDto, staffId, isStaff)` in `PaymentService.java:L59-L99`.
  - Supports multiple methods: `CASH`, `CARD`, `ONLINE`, `BANK_TRANSFER`.
  - Generates automated unique reference code: `"TXN-" + System.currentTimeMillis()`.
  - Instant payments (Card / Online) are marked `VERIFIED` and automatically trigger `invoiceRepository.reduceOutstandingBalance(...)`.
  - Bank Transfer payments are placed into `PENDING` status, requiring staff slip verification.
- **READ:** `getInvoiceByReservation(...)` delivers invoice statement and balance; `getPendingVerifications()` displays the bank-transfer approval queue for finance staff.
- **UPDATE (Bank Slip Verification):** `verifyBankTransferReceipt(paymentId, approve, staffId)` in `PaymentService.java:L102-L119`. On approval, sets status to `VERIFIED` and deducts balance; on rejection, sets status to `FAILED`.
- **DELETE:** `deletePayment(...)` and `deleteInvoice(...)` with staff audit logging in `ActivityLog`.
- **Complex Logic:** Real-time outstanding balance calculation and dual-state asynchronous bank transfer verification pipeline.

2. Data Validations (Frontend & Backend)

- **Frontend (`booking.js` & `checkout.html`):** Reveals bank account details only when `BANK_TRANSFER` is selected; deposit input maximum clamped to total invoice amount.
- **Backend (`PaymentRequestDto.java` & `PaymentService.java`):** Amount must be non-null and strictly greater than zero (`amount.compareTo(BigDecimal.ZERO) > 0`); verification rejected if payment is not in `PENDING` status.
- **Database Constraints:** `CHECK (method IN ('CASH','CARD','ONLINE','BANK_TRANSFER'))`, `CHECK (status IN ('PENDING','VERIFIED','FAILED'))`.

3. Database Schema & Tables

- `Invoice` (PK: `invoice_id`, Unique FK: `reservation_id`, `total_amount`, `outstanding_balance`, `issue_date`).
- `Payment` (PK: `payment_id`, FK: `reservation_id`, FK: `processed_by_staff_id`, `method`, `amount`, `reference_number`, `status`).

4. Design Patterns & Justification

- **DAO / Repository Pattern:** Encapsulates SQL updates (such as atomic balance decrements) inside `InvoiceRepository` and `PaymentRepository`.
- **Audit Logging:** Integration with `ActivityLogService` guarantees non-repudiation for financial transactions.

FUNCTION 4: CUSTOMER (GUEST) MANAGEMENT

Assigned Member: IT25100457 – Perera.P.P.S | **Main Users:** Guest, Customer Relations Executive

1. CRUD Operations & Complex Business Logic

- **Primary Files:** `com/smartstay/controller/AuthController.java`, `com/smartstay/service/AuthService.java`, `com/smartstay/service/GuestService.java`, `com/smartstay/repository/GuestRepository.java`
- **CREATE (Self-Registration):** `registerGuest(RegisterGuestDto)` in `AuthService.java:L85-L107`. Validates matching password confirmation, checks email uniqueness in database, and securely hashes password using Spring Security's **BCrypt**.

```
guest.setPassword(passwordEncoder.encode(dto.getPassword()));
int guestId = guestRepository.save(guest);
```

- **READ (Itinerary & Profile):** `guestOpt = guestRepository.findByEmail(email)` during login; retrieves guest stay history and active itineraries in `templates/reservations/my-reservations.html`.
- **UPDATE (Profile & Password):** Updates contact information (phone, address) and allows password resets via BCrypt rehashing.
- **DELETE:** IT administrator can deactivate or remove guest accounts through the administrative control panel.
- **Complex Logic:** Secure password hashing with BCrypt salt generation, session creation, and duplicate identity prevention.

2. Data Validations (Frontend & Backend)

- **Frontend (`validation.js`):** Validates password matching confirmation, minimum 6 characters length, and standard email pattern format regex.
- **Backend (`RegisterGuestDto.java`):** Annotated with `@NotBlank`, `@Email`, `@Size(min=2, max=100)`, `@Size(min=6)`. Service layer throws `ValidationException("An account with this email already exists")` if email is duplicated.
- **Database Constraints:** `UNIQUE (email)` constraint on `Guest` table guarantees data integrity at the database engine level.

3. Database Schema & Tables

- `Guest` (PK: `guest_id`, `name`, `email` [UNIQUE], `phone`, `address`, `password` [BCrypt Hash]).
- Linked to child records: `Reservation(guest_id)` and `Feedback(guest_id)`.

4. Design Patterns & Justification

- **DTO Pattern:** `RegisterGuestDto` decouples HTTP registration payload from the internal domain model `Guest`, preventing mass-assignment attacks.
- **Service Layer Pattern:** Centralizes authentication, password matching, and session token creation in `AuthService`.

FUNCTION 5: FEEDBACK AND COMPLAINT MANAGEMENT

Assigned Member: IT25100464 – Wijesiri.W.D.H.A.S | **Main Users:** Guest, Customer Relations Executive, General Manager

1. CRUD Operations & Complex Business Logic

- **Primary Files:** `com/smartstay/controller/FeedbackController.java`, `com/smartstay/service/FeedbackService.java`, `com/smartstay/repository/FeedbackRepository.java`, `com/smartstay/repository/RefundRepository.java`
- **CREATE (Submit Review/Complaint):** `submitFeedback(...)` in `FeedbackService.java:L55-L68` captures star ratings (1–5), issue categories (`SERVICE`, `CLEANLINESS`, `BILLING`, `CANCELLATION`), and initial status `OPEN`.
- **READ:** `getAllFeedback()` and `getFeedbackByGuest()` automatically joins any associated `Refund` payout tickets.
- **UPDATE (Resolution Lifecycle):** `updateFeedbackStatus(feedbackId, status, staffId)` manages transitions (`OPEN` → `IN_PROGRESS` → `RESOLVED`); `addStaffResponse(...)` records official hotel responses.
- **DELETE:** `deleteFeedback(...)` safely removes linked `Refund` records before deleting the feedback ticket to preserve foreign key constraints.
- **Complex Logic (Policy-Based Refund Workflow):**
  - Customer Relations issues refund ticket via `issueRefundForFeedback(...)` (`FeedbackService.java:L120-L138`) setting status to `PENDING`.
  - General Manager reviews and issues decision via `processRefundDecision(...)` (`APPROVED`, `REJECTED`, `COMPLETED`).
  - Authorization guardrail: `editFeedback(...)` prevents guests from modifying reviews submitted by other users.

2. Data Validations (Frontend & Backend)

- **Frontend (`validation.js`):** Validates that star rating is chosen between 1 and 5; prevents submission of blank or whitespace comments.
- **Backend (`FeedbackService.java`):** Validates positive refund amount (`amount > 0`); validates user identity ownership (`f.getGuestId().equals(guestId)`).
- **Database Constraints:** `CHECK (rating BETWEEN 1 AND 5)`, `CHECK (status IN ('OPEN','IN_PROGRESS','RESOLVED'))`, `CHECK (status IN ('PENDING','APPROVED','REJECTED','COMPLETED'))` on Refund.

3. Database Schema & Tables

- `Feedback` (PK: `feedback_id`, FK: `guest_id`, FK: `reservation_id`, FK: `handled_by_staff_id`, `rating`, `category`, `comments`, `status`).
- `Refund` (PK: `refund_id`, FK: `feedback_id`, `amount`, `reason`, `status`).

4. Design Patterns & Justification

- **State Machine Pattern:** Enforces distinct ticket lifecycles (`OPEN` → `IN_PROGRESS` → `RESOLVED`) and refund authorization workflows.
- **Cascade Preservation:** Clean repository encapsulation avoids orphaned financial records.

FUNCTION 6: USER AND STAFF MANAGEMENT

Assigned Member: IT25100470 – FernandoB.V.I | **Main Users:** IT Coordinator, General Manager, All Staff Roles

1. CRUD Operations & Complex Business Logic

- **Primary Files:** `com/smartstay/controller/StaffController.java`, `com/smartstay/service/StaffService.java`, `com/smartstay/config/AuthInterceptor.java`, `com/smartstay/service/ActivityLogService.java`
- **CREATE:** `StaffService.createStaff(...)` creates staff accounts with role assignment (`MANAGER`, `RESERVATIONS`, `FRONT_OFFICE`, `GUEST_RELATIONS`, `FINANCE`, `IT`) and BCrypt password encryption.
- **READ (Directory & Security Audit Trail):** Staff directory listing; `ActivityLogRepository.findAll()` displays system-wide audit records with timestamps, staff IDs, and detailed descriptions of actions performed.
- **UPDATE:** Staff account updates and status toggling (`ACTIVE` vs. `DEACTIVATED`).
- **DELETE (Soft Deactivation):** IT Coordinator deactivates accounts (rather than hard deletion) to preserve historical foreign key integrity on past bookings and financial audits.
- **Complex Logic (Centralized Role-Based Access Control):**
  - Implemented via Spring MVC **HandlerInterceptor** in `AuthInterceptor.java:L17-L87`.
  - Inspects `UserSession` on every HTTP request and verifies permissions before allowing access:

```
// AuthInterceptor RBAC Enforcement Snippet:
if (path.startsWith("/staff/it") && !user.isIt() && !user.isManager()) {
    response.sendRedirect("/dashboard?error=Access Denied: IT Coordinator role required");
    return false;
}
```

  - Deactivated accounts are immediately blocked from logging into the system during `AuthService.login()`.

2. Data Validations (Frontend & Backend)

- **Frontend:** Form validation for required staff role selection, branch assignment, and valid email format.
- **Backend:** Verifies staff email uniqueness; blocks login for `DEACTIVATED` accounts; interceptor validates role permissions for all protected administrative URLs.
- **Database Constraints:** `CHECK (role IN ('MANAGER','RESERVATIONS','FRONT_OFFICE','GUEST_RELATIONS','FINANCE','IT'))`, `CHECK (status IN ('ACTIVE','DEACTIVATED'))`, `UNIQUE (email)`.

3. Database Schema & Tables

- `Staff` (PK: `staff_id`, FK: `branch_id`, Self-referencing FK: `supervisor_id`, `name`, `email`, `password`, `role`, `status`).
- `ActivityLog` (PK: `log_id`, FK: `staff_id` → `Staff`, `action`, `timestamp`).

4. Design Patterns & Justification

- **Interceptor / Front Controller Pattern:** Centralizes authentication and RBAC checks across the entire web application, eliminating repetitive security code inside individual controller methods.
- **Singleton Pattern:** Spring-managed service and interceptor beans are singletons, ensuring high throughput and optimal memory usage.